

Условия и логика

Научить программу принимать решения через if / elif / else и комбинировать условия логическими операторами.

DAY 09

EXPRESSION

comparison

logical operator

if

elif

ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

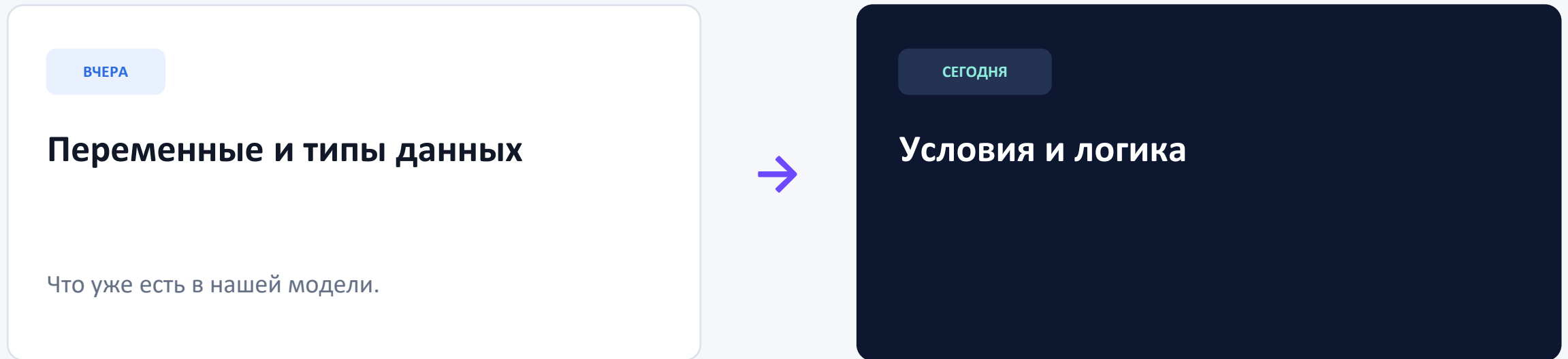
Конкретные действия вместо списка терминов.

- 01 Научить программу принимать решения через if / elif / else и комбинировать условия логическими операторами.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.



Вчера: Переменные и типы данных. Сегодня мы добавляем следующий слой: условия и логика.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

**Без условий программа всегда делает одно и то же.
Условия превращают данные пользователя в разные
сценарии поведения.**

Вопрос себе

**Какую реальную проблему
решает сегодняшняя тема?**

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

if — охранник у двери: он проверяет правило и пропускает выполнение только когда условие истинно.

ПЕРЕВОД В IT

expression

часть кода, которая вычисляется в значение

comparison

проверка отношения между двумя значениями

logical operator

оператор and, or или not для объединения условий

Слой 1: expression • comparison • logical operator

Сначала смысл, затем профессиональный термин.

**expression**

часть кода, которая вычисляется в значение

**comparison**

проверка отношения между двумя значениями

**logical operator**

оператор and, or или not для объединения условий

Слой 2: if • elif • else

Сначала смысл, затем профессиональный термин.

**if**

ветка, выполняемая при истинном условии

**elif**

дополнительная проверка после if

**else**

ветка для всех не подошедших условий

Слой 3: condition • branch

Сначала смысл, затем профессиональный термин.

condition

проверка, результат которой истина или ложь

branch

отдельный путь выполнения программы

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: **truthy** • **short-circuit**

Сначала смысл, затем профессиональный термин.

truthy

значение, воспринимаемое как истина

short-circuit

остановка вычисления логического выражения раньше конца

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day09_step1

```
age = 18
if age >= 18:
    print("Доступ разрешён")
else:
    print("Доступ запрещён")
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
age = 18
```

Сохраняем значение или результат вычисления под понятным именем.

```
if age >= 18:
```

Проверяем условие: ветка выполнится только при True.

```
print("Доступ разрешён")
```

Показываем результат, чтобы увидеть состояние программы.

```
else:
```

Python выполняет эту строку сверху вниз.

```
print("Доступ запрещён")
```

Показываем результат, чтобы увидеть состояние программы.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
age = 18
if age >= 18:
    print("Доступ разрешён")
else:
    print("Доступ запрещён")
```

РЕЗУЛЬТАТ

Доступ разрешён

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.

```
age = 16
has_parent = True
if age >= 18 or has_parent:
    print("Можно войти")
```

changed



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

Можно войти

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.

```
age = 20
if age >= 18
    print("OK")
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

После условия if нужен двоеточие. Оно сообщает Python, что дальше начинается блок кода этой ветки.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

ИСПРАВЛЕНИЕ

Исправленный вариант

Меняем только то, что действительно было причиной.

```
age = 20
if age >= 18:
    print("OK")
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Определить возрастную категорию: ребёнок, подросток или взрослый.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Берём возраст.**
- 2 Сначала проверяем самое маленькое ограничение.
- 3 Если оно не подошло — проверяем следующий диапазон.
- 4 Иначе остаётся взрослая категория.
- 5 Выполняется только одна ветка.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
age = 15
if age < 13:
    category = "ребёнок"
elif age < 18:
    category = "подросток"
```

solution_part1

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



```
age = 15
if age < 13:
    category = "ребёнок"
elif age < 18:
    category = "подросток"
else:
    category = "взрослый"
print(category)
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

подросток

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Рассчитай стоимость доставки: бесплатно при сумме от 50, иначе 5.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Сначала сравни `total` с 50, затем присвой `delivery` одно из двух значений.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
total = 42
if total >= 50:
    delivery = 0
else:
    delivery = 5
print(delivery)
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

5

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

ЗАДАНИЕ

Сделай проверку входа: правильный логин И правильный пароль должны дать доступ.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



mini_project

```
login = "admin"
password = "python"
if login == "admin" and password == "python":
    print("Вход выполнен")
else:
    print("Неверные данные")
```

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

Вход выполнен

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

Забывать двоеточие.

02

Нарушать отступы блока.

03

Путать = и ==.

04

Строить пересекающиеся elif-условия без понимания порядка.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «expression» и какую проблему это понятие помогает описать?

2

Чем «expression» отличается от «if»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **expression — часть кода, которая вычисляется в значение.**

2 expression: часть кода, которая вычисляется в значение. if: ветка, выполняемая при истинном условии.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 **Доступ разрешён**

4 **После условия if нужен двоеточие. Оно сообщает Python, что дальше начинается блок кода этой ветки.**

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Научить программу принимать решения через `if / elif / else` и комбинировать условия логическими операторами.
- ✓ Читать и объяснять пример темы «Условия и логика».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

[ДАЛЬШЕ](#)

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Циклы

Завтра: Циклы. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.